

North Slope Data Layer Map

April 12, 2026

```
[1]: # =====
# 0. IMPORTS
# =====
from pathlib import Path
import geopandas as gpd
import pandas as pd
import numpy as np
import plotly.graph_objects as go

# =====
# 1. LOAD FEATURE PARQUETS
# =====
base_dir = Path.cwd()

seis2d = gpd.read_parquet(base_dir / "clean_2d_seismic_features.parquet")
seis3d = gpd.read_parquet(base_dir / "clean_3d_seismic_features.parquet")
wells = gpd.read_parquet(base_dir / "clean_well_locations_features_dedup.
↳parquet")
assessment = gpd.read_parquet(base_dir / "north_slope_assessment_units_features.
↳parquet")
extent = gpd.read_parquet(base_dir / "north_slope_extent_features.parquet")

print("Loaded:")
print(" 2D seismic:", seis2d.shape)
print(" 3D seismic:", seis3d.shape)
print(" wells:", wells.shape)
print(" assessment:", assessment.shape)
print(" extent:", extent.shape)

# =====
# 2. KEEP ONLY NORTH SLOPE WELLS FOR THIS MAP
# =====
extent_union = extent.to_crs("EPSG:4326").geometry.union_all()
wells_ns = wells[wells.geometry.within(extent_union)].copy()

print("\nNorth Slope wells only:", wells_ns.shape)
```

```

# =====
# 3. HELPER COLUMNS
# =====
# Clean text columns for hover
for df, cols in [
    (seis2d, ["survey_name", "company", "type"]),
    (seis3d, ["survey_name", "company", "survey_type", "type"]),
    (wells_ns, ["operator", "field", "type", "au_name"]),
    (assessment, ["au_name", "province", "tps_name", "type"]),
]:
    for c in cols:
        if c in df.columns:
            df[c] = df[c].astype("string").fillna("NA")

# Distance bins for wells
wells_ns["dist_2d_bin"] = pd.cut(
    wells_ns["dist_to_2d_km"],
    bins=[-np.inf, 1, 5, 10, np.inf],
    labels=["<1 km", "1-5 km", "5-10 km", ">10 km"]
).astype("string")

# Well type mode
wells_ns["well_class"] = np.where(wells_ns["is_exploratory"], "Exploratory",
    ↪ "Other")

# In 3D mode
wells_ns["coverage_class"] = np.where(wells_ns["in_3d"], "Inside 3D", "Outside_
    ↪ 3D")

# 3D survey age bins
if "survey_age_years" in seis3d.columns:
    seis3d["age_bin"] = pd.cut(
        seis3d["survey_age_years"],
        bins=[-np.inf, 5, 10, 15, np.inf],
        labels=["0-5 yrs", "6-10 yrs", "11-15 yrs", "16+ yrs"]
    ).astype("string")

# 2D survey age bins
if "survey_age_years" in seis2d.columns:
    seis2d["age_bin"] = pd.cut(
        seis2d["survey_age_years"],
        bins=[-np.inf, 5, 10, 15, np.inf],
        labels=["0-5 yrs", "6-10 yrs", "11-15 yrs", "16+ yrs"]
    ).astype("string")

# =====
# 4. GEOMETRY HELPERS

```

```

# =====
def line_parts(geom):
    """Return list of coordinate sequences for LineString or MultiLineString."""
    if geom is None or geom.is_empty:
        return []
    gtype = geom.geom_type
    if gtype == "LineString":
        x, y = geom.xy
        return [list(zip(x, y))]
    elif gtype == "MultiLineString":
        parts = []
        for part in geom.geoms:
            x, y = part.xy
            parts.append(list(zip(x, y)))
        return parts
    return []

def polygon_rings(geom):
    """Return exterior rings for Polygon or MultiPolygon."""
    if geom is None or geom.is_empty:
        return []
    gtype = geom.geom_type
    rings = []
    if gtype == "Polygon":
        x, y = geom.exterior.xy
        rings.append(list(zip(x, y)))
    elif gtype == "MultiPolygon":
        for poly in geom.geoms:
            x, y = poly.exterior.xy
            rings.append(list(zip(x, y)))
    return rings

def rings_to_lonlat_lists(rings):
    """Convert multiple rings into lon/lat lists separated by None."""
    lons, lats = [], []
    for ring in rings:
        for lon, lat in ring:
            lons.append(lon)
            lats.append(lat)
        lons.append(None)
        lats.append(None)
    return lons, lats

# =====
# 5. CREATE FIGURE
# =====
fig = go.Figure()

```

```

trace_groups = {
    "assessment": [],
    "seis3d": [],
    "seis2d": [],
    "wells_mode1": [], # exploratory
    "wells_mode2": [], # in_3d
    "wells_mode3": [], # distance bin
}

# =====
# 6. ADD NORTH SLOPE EXTENT OUTLINE
# =====
for _, row in extent.iterrows():
    rings = polygon_rings(row.geometry)
    lons, lats = rings_to_lonlat_lists(rings)

    fig.add_trace(
        go.Scattermapbox(
            lon=lons,
            lat=lats,
            mode="lines",
            name="North Slope Extent",
            line=dict(width=3, color="black"),
            hoverinfo="skip",
            showlegend=True
        )
    )

extent_trace_idx = 0

# =====
# 7. ADD ASSESSMENT UNIT OUTLINES
# =====
au_colors = {
    "Brookian Topset": "#1f77b4",
    "Brookian Foreset-Bottomset": "#ff7f0e",
    "Beaufortian Strata North": "#2ca02c",
    "Beaufortian Strata South": "#d62728",
    "Ellesmerian Strata North": "#9467bd",
    "Ellesmerian Strata South": "#8c564b",
}

for _, row in assessment.iterrows():
    rings = polygon_rings(row.geometry)
    lons, lats = rings_to_lonlat_lists(rings)
    color = au_colors.get(row["au_name"], "#555555")

```

```

fig.add_trace(
    go.Scattermapbox(
        lon=lons,
        lat=lats,
        mode="lines",
        name=f'AU: {row["au_name"]}',
        line=dict(width=2, color=color),
        hovertemplate=(
            f"<b>{row['au_name']}</b><br>"
            f"Province: {row['province']}<br>"
            f"TPS: {row['tps_name']}<br>"
            f"Area km²: {row['area_km2']:.1f}<extra></extra>"
        ),
        showlegend=True
    )
)
trace_groups["assessment"].append(len(fig.data) - 1)

# =====
# 8. ADD 3D SEISMIC POLYGONS
# =====
for _, row in seis3d.iterrows():
    rings = polygon_rings(row.geometry)
    lons, lats = rings_to_lonlat_lists(rings)

    fig.add_trace(
        go.Scattermapbox(
            lon=lons,
            lat=lats,
            mode="lines",
            name="3D seismic",
            line=dict(width=2, color="rgba(0, 0, 255, 0.55)"),
            fill="toself",
            fillcolor="rgba(0, 0, 255, 0.10)",
            hovertemplate=(
                f"<b>{row['survey_name']}</b><br>"
                f"Company: {row['company']}<br>"
                f"Survey type: {row['survey_type']}<br>"
                f"Year acquired: {row['year_acquired']}<br>"
                f"Year released: {row['year_released']}<br>"
                f"Square miles (orig): {row['square_miles']:.2f}<br>"
                f"Area km² (geom): {row['area_km2']:.2f}<br>"
                f"Survey age: {row['survey_age_years']:.0f} years<extra></
<extra>"
            ),
            showlegend=False
        )
    )

```

```

    )
)
trace_groups["seis3d"].append(len(fig.data) - 1)

# Add one legend-only entry for 3D
fig.add_trace(
    go.Scattermapbox(
        lon=[None],
        lat=[None],
        mode="lines",
        name="3D Seismic",
        line=dict(width=3, color="rgba(0, 0, 255, 0.7)"),
        showlegend=True,
        hoverinfo="skip"
    )
)
legend_3d_idx = len(fig.data) - 1

# =====
# 9. ADD 2D SEISMIC LINES
# =====
for _, row in seis2d.iterrows():
    parts = line_parts(row.geometry)

    all_lons, all_lats = [], []
    for part in parts:
        for lon, lat in part:
            all_lons.append(lon)
            all_lats.append(lat)
        all_lons.append(None)
        all_lats.append(None)

    fig.add_trace(
        go.Scattermapbox(
            lon=all_lons,
            lat=all_lats,
            mode="lines",
            name="2D seismic",
            line=dict(width=2, color="rgba(255, 100, 0, 0.65)"),
            hovertemplate=(
                f"<b>{row['survey_name']}</b><br>"
                f"Company: {row['company']}<br>"
                f"Year acquired: {row['year_acquired']}<br>"
                f"Year released: {row['year_released']}<br>"
                f"Line miles (orig): {row['line_miles']:.2f}<br>"
                f"Length km (geom): {row['length_km']:.2f}<br>"
            )
        )

```

```

        f"Survey age: {row['survey_age_years']:.0f} years<extra></
<extra>"
    ),
    showlegend=False
)
)
trace_groups["seis2d"].append(len(fig.data) - 1)

# Add one legend-only entry for 2D
fig.add_trace(
    go.Scattermapbox(
        lon=[None],
        lat=[None],
        mode="lines",
        name="2D Seismic",
        line=dict(width=3, color="rgba(255, 100, 0, 0.8)"),
        showlegend=True,
        hoverinfo="skip"
    )
)
legend_2d_idx = len(fig.data) - 1

# =====
# 10. WELL TRACES - MODE 1: EXPLORATORY VS OTHER
# =====
model_colors = {
    "Exploratory": "red",
    "Other": "gold",
}

for cat, color in model_colors.items():
    sub = wells_ns[wells_ns["well_class"] == cat].copy()

    fig.add_trace(
        go.Scattermapbox(
            lon=sub["lon"],
            lat=sub["lat"],
            mode="markers",
            name=f"Wells: {cat}",
            marker=dict(size=7, color=color, opacity=0.75),
            customdata=np.stack([
                sub["operator"].astype(str),
                sub["field"].astype(str),
                sub["au_name"].astype(str),
                sub["dist_to_2d_km"].round(2).astype(str),
                sub["in_3d"].astype(str),
            ], axis=1),

```

```

        hovertemplate=(
            "<b>{%{customdata[1]}}</b><br>"
            "Operator: {%{customdata[0]}}<br>"
            "AU: {%{customdata[2]}}<br>"
            "Dist to 2D (km): {%{customdata[3]}}<br>"
            "In 3D: {%{customdata[4]}}<extra></extra>"
        ),
        visible=True
    )
)
trace_groups["wells_model1"].append(len(fig.data) - 1)

# =====
# 11. WELL TRACES - MODE 2: IN 3D VS OUTSIDE
# =====
mode2_colors = {
    "Inside 3D": "cyan",
    "Outside 3D": "gray",
}

for cat, color in mode2_colors.items():
    sub = wells_ns[wells_ns["coverage_class"] == cat].copy()

    fig.add_trace(
        go.Scattermapbox(
            lon=sub["lon"],
            lat=sub["lat"],
            mode="markers",
            name=f"Wells: {cat}",
            marker=dict(size=7, color=color, opacity=0.75),
            customdata=np.stack([
                sub["operator"].astype(str),
                sub["field"].astype(str),
                sub["au_name"].astype(str),
                sub["dist_to_2d_km"].round(2).astype(str),
                sub["in_3d"].astype(str),
            ], axis=1),
            hovertemplate=(
                "<b>{%{customdata[1]}}</b><br>"
                "Operator: {%{customdata[0]}}<br>"
                "AU: {%{customdata[2]}}<br>"
                "Dist to 2D (km): {%{customdata[3]}}<br>"
                "In 3D: {%{customdata[4]}}<extra></extra>"
            ),
            visible=False
        )
    )
)

```



```

        trace_groups["wells_mode2"].append(len(fig.data) - 1)

# =====
# 12. WELL TRACES - MODE 3: DISTANCE TO 2D BINS
# =====
mode3_colors = {
    "<1 km": "lime",
    "1-5 km": "yellow",
    "5-10 km": "orange",
    ">10 km": "purple",
}

for cat, color in mode3_colors.items():
    sub = wells_ns[wells_ns["dist_2d_bin"] == cat].copy()

    fig.add_trace(
        go.Scattermapbox(
            lon=sub["lon"],
            lat=sub["lat"],
            mode="markers",
            name=f"Wells: {cat}",
            marker=dict(size=7, color=color, opacity=0.75),
            customdata=np.stack([
                sub["operator"].astype(str),
                sub["field"].astype(str),
                sub["au_name"].astype(str),
                sub["dist_to_2d_km"].round(2).astype(str),
                sub["in_3d"].astype(str),
            ], axis=1),
            hovertemplate=(
                "<b>{%customdata[1]}</b><br>"
                "Operator: {%customdata[0]}<br>"
                "AU: {%customdata[2]}<br>"
                "Dist to 2D (km): {%customdata[3]}<br>"
                "In 3D: {%customdata[4]}<extra></extra>"
            ),
            visible=False
        )
    )
    trace_groups["wells_mode3"].append(len(fig.data) - 1)

# =====
# 13. VISIBILITY HELPERS
# =====
n_traces = len(fig.data)

def vis_for_mode(mode_name):

```

```

vis = [True] * n_traces # keep non-well layers visible by default

# turn off all well traces first
for idx in trace_groups["wells_mode1"] + trace_groups["wells_mode2"] +
↪trace_groups["wells_mode3"]:
    vis[idx] = False

# turn on selected mode
for idx in trace_groups[mode_name]:
    vis[idx] = True

return vis

vis_model1 = vis_for_mode("wells_mode1")
vis_model2 = vis_for_mode("wells_mode2")
vis_model3 = vis_for_mode("wells_mode3")

# =====
# 14. MAP CENTER
# =====
center_lon = wells_ns["lon"].median()
center_lat = wells_ns["lat"].median()

# =====
# 15. LAYOUT
# =====
fig.update_layout(
    title="North Slope: Wells, 2D/3D Seismic, and Assessment Units",
    height=900,
    mapbox=dict(
        style="carto-positron",
        zoom=5.4,
        center=dict(lat=center_lat, lon=center_lon),
    ),
    margin=dict(l=10, r=10, t=60, b=10),
    legend=dict(
        title="Layers / Active Well Colors",
        orientation="v",
        yanchor="top",
        y=0.99,
        xanchor="left",
        x=0.01,
        bgcolor="rgba(255,255,255,0.85)"
    ),
    updatemenus=[
        dict(
            type="dropdown",

```

```

        direction="down",
        x=0.01,
        y=1.08,
        xanchor="left",
        yanchor="top",
        buttons=[
            dict(
                label="Well colors: Exploratory vs Other",
                method="update",
                args=[
                    {"visible": vis_model},
                    {"title": "North Slope: Wells colored by Exploratory vs_
↪Other"}
                ]
            ),
            dict(
                label="Well colors: In 3D vs Outside",
                method="update",
                args=[
                    {"visible": vis_mode2},
                    {"title": "North Slope: Wells colored by 3D Seismic_
↪Coverage"}
                ]
            ),
            dict(
                label="Well colors: Distance to 2D",
                method="update",
                args=[
                    {"visible": vis_mode3},
                    {"title": "North Slope: Wells colored by Distance to 2D_
↪Seismic"}
                ]
            ),
        ],
        showactive=True
    ),
]
)

# =====
# 16. SHOW
# =====
fig.show()

# =====
# 17. OPTIONAL: SAVE HTML
# =====

```

```
out_html = base_dir / "north_slope_plotly_advanced.html"
fig.write_html(out_html, include_plotlyjs="cdn")
print(f"Saved interactive HTML to: {out_html.resolve()}")
```

Loaded:

```
2D seismic: (8, 13)
3D seismic: (24, 15)
wells: (9861, 24)
assessment: (6, 10)
extent: (1, 2)
```

North Slope wells only: (8066, 24)

/tmp/ipykernel_646/3573083751.py:146: DeprecationWarning:

scattermapbox is deprecated! Use *scattermap* instead. Learn more at:
<https://plotly.com/python/mapbox-to-maplibre/>

/tmp/ipykernel_646/3573083751.py:177: DeprecationWarning:

scattermapbox is deprecated! Use *scattermap* instead. Learn more at:
<https://plotly.com/python/mapbox-to-maplibre/>

/tmp/ipykernel_646/3573083751.py:202: DeprecationWarning:

scattermapbox is deprecated! Use *scattermap* instead. Learn more at:
<https://plotly.com/python/mapbox-to-maplibre/>

/tmp/ipykernel_646/3573083751.py:227: DeprecationWarning:

scattermapbox is deprecated! Use *scattermap* instead. Learn more at:
<https://plotly.com/python/mapbox-to-maplibre/>

/tmp/ipykernel_646/3573083751.py:254: DeprecationWarning:

scattermapbox is deprecated! Use *scattermap* instead. Learn more at:
<https://plotly.com/python/mapbox-to-maplibre/>

/tmp/ipykernel_646/3573083751.py:276: DeprecationWarning:

scattermapbox is deprecated! Use *scattermap* instead. Learn more at:
<https://plotly.com/python/mapbox-to-maplibre/>

/tmp/ipykernel_646/3573083751.py:300: DeprecationWarning:

scattermapbox is deprecated! Use *scattermap* instead. Learn more at:
<https://plotly.com/python/mapbox-to-maplibre/>

```
/tmp/ipykernel_646/3573083751.py:300: DeprecationWarning:

*scattermapbox* is deprecated! Use *scattermap* instead. Learn more at:
https://plotly.com/python/mapbox-to-maplibre/

/tmp/ipykernel_646/3573083751.py:337: DeprecationWarning:

*scattermapbox* is deprecated! Use *scattermap* instead. Learn more at:
https://plotly.com/python/mapbox-to-maplibre/

/tmp/ipykernel_646/3573083751.py:337: DeprecationWarning:

*scattermapbox* is deprecated! Use *scattermap* instead. Learn more at:
https://plotly.com/python/mapbox-to-maplibre/

/tmp/ipykernel_646/3573083751.py:376: DeprecationWarning:

*scattermapbox* is deprecated! Use *scattermap* instead. Learn more at:
https://plotly.com/python/mapbox-to-maplibre/

/tmp/ipykernel_646/3573083751.py:376: DeprecationWarning:

*scattermapbox* is deprecated! Use *scattermap* instead. Learn more at:
https://plotly.com/python/mapbox-to-maplibre/

/tmp/ipykernel_646/3573083751.py:376: DeprecationWarning:

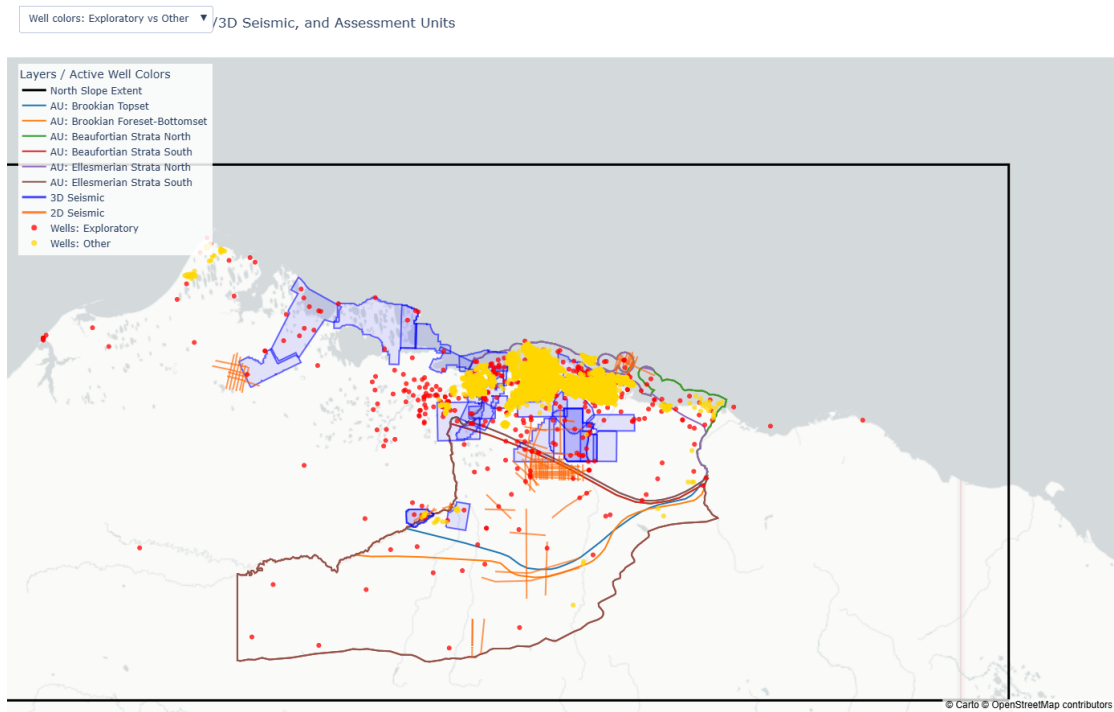
*scattermapbox* is deprecated! Use *scattermap* instead. Learn more at:
https://plotly.com/python/mapbox-to-maplibre/

/tmp/ipykernel_646/3573083751.py:376: DeprecationWarning:

*scattermapbox* is deprecated! Use *scattermap* instead. Learn more at:
https://plotly.com/python/mapbox-to-maplibre/

/tmp/ipykernel_646/3573083751.py:376: DeprecationWarning:

*scattermapbox* is deprecated! Use *scattermap* instead. Learn more at:
https://plotly.com/python/mapbox-to-maplibre/
```



Saved interactive HTML to:
/home/jovyan/notebooks/north_slope_plotly_advanced.html

[]: